

ArUco Pose Logger and Filter Analysis

29 July 2026

Purpose: record numerical ArUco poses for filter analysis. The logger does not record video, does not run OpenCV a second time, and does not add CSV logic to `main_3d.gd`.

1. Input and output

Paul's C++ code produces one dictionary for every processed camera image:

```
result[ids[i]] = lens_pose * Transform3D(basis, origin);
```

Conceptually:

```
{
  0: Transform3D(...), # common
  1: Transform3D(...), # chest
  2: Transform3D(...)  # navel
}
```

One completed dictionary becomes one CSV row:

```
Quest image
↓
Paul/OpenCV calculates marker 6-DoF
↓
Dictionary {marker ID: Transform3D}
↓
Standalone logger copies the result
↓
One CSV row
```

The CSV stores logger time, detected count, camera pose, and the three marker poses. Each pose contains position `x`, `y`, `z` in metres and quaternion rotation `qx`, `qy`, `qz`, `qw`. A missing marker receives `seen=0` and empty pose fields.

2. Logger code structure

Section	Function	Responsibility
Settings	constants and exports	Duration, marker IDs, node references
Start	<code>_ready()</code> , <code>_on_button()</code>	Connect right-controller A
Main flow	<code>_process()</code>	Copy, check, and save the result
Safe copy	<code>_get_latest_opencv_result()</code>	Copy Paul's result under the mutex
Duplicate check	<code>_is_new_result()</code>	Write each changed result once
File lifecycle	<code>_start_recording()</code> , <code>_stop_recording()</code>	Open and close CSV
CSV conversion	<code>_save_result_as_csv_row()</code>	Convert transforms into columns

```
func _process(_delta: float) -> void:
    var result := _get_latest_opencv_result()
    if result.is_empty():
        return

    if not _is_new_result(result):
        return

    if _recording:
        _save_result_as_csv_row(result.markers, result.camera_pose)
```

3. Thread safety and duplicate prevention

OpenCV works on a worker thread. The mutex prevents the worker from replacing the dictionary halfway through the copy:

```
result_mutex.lock()
var markers = detection_source.get("_result_markers").duplicate(true)
var camera_pose = detection_source.get("_result_cam_xform")
result_mutex.unlock()
```

Godot calls `_process()` about 72 times per second, while OpenCV normally produces 8–12 results per second. A hash fingerprints the latest result, preventing the same stored dictionary from being written on several Godot frames:

```
var result_hash := hash(result)
if result_hash == _last_result_hash:
    return
```

4. Experimental procedure

1. Launch the Quest application.
2. Make the required markers visible.
3. Press A once on the right controller.
4. Maintain the experimental condition for 30 seconds.
5. The logger stops automatically.
6. Do not press B during raw logging; B performs calibration.

Files are stored as `user://aruco_raw_<timestamp>.csv`.

5. Stationary-test analysis method

The stationary recording contained 260 OpenCV samples over 29.8 seconds. All three markers were simultaneously detected in 234 samples.

Robust offsets were calculated for every synchronized sample:

```
common_to_navel = common_pose.affine_inverse() * navel_pose
chest_to_navel   = chest_pose.affine_inverse()  * navel_pose
```

The navel was reconstructed from ID 0 and ID 1, fused, and compared with the robust stationary centre.

Fused ID 0 + ID 1 error	Position	Rotation
Median	0.81 mm	0.19°
95th percentile	2.63 mm	0.46°
99th percentile	7.55 mm	2.06°
Maximum	9.60 mm	2.46°

6. Filter interpretation

Parameter	Current	Evidence-based starting value
Position dead zone	15 mm	4–5 mm after outlier handling
Rotation dead zone	3°	approximately 1°
Smoothing time	0.40 seconds	Requires a controlled movement test

The stationary log measures noise and can suggest dead zones. It cannot measure tracking delay, so it cannot determine the final smoothing time.

7. Conclusions

- Normal fused common/chest noise is a few millimetres and less than one degree.
- There is no evidence of continuously accumulating relative drift.
- ID 2 produced large errors after detection returned; calibration should use several stable samples.
- ID 2 should be used for calibration and removed from normal common/chest fusion.
- Position and rotation should be filtered independently.
- A controlled movement log is required to select smoothing time.

8. Short oral explanation

Paul's OpenCV code produces one dictionary of marker poses for each processed camera image. I added a separate logger node that safely copies this dictionary using the same mutex. A hash prevents the same stored result from being written more than once. When I press A, each new dictionary is converted into one CSV row for 30 seconds. The CSV contains positions and quaternion rotations for the common, chest, and navel markers. No video is recorded, and the original detection script remains separate.

9. Relevant files

- `project/aruco_csv_logger.gd`
- `project/main_3d.tscn`
- `src/OpenCVProcessor.cpp`
- `analyze_static_aruco.py`
- `aruco_raw_1785317240_01.csv`